

RouterSploit模块开发指南

1. 实验目标

通过本实验，学习者将：

- 掌握RouterSploit模块开发的基本规范和要求
- 学习如何编写符合RouterSploit框架的漏洞利用模块
- 理解RouterSploit模块的继承关系和接口设计
- 掌握模块测试和调试方法

实验要求：

- 在阅读学习完整个实验手册中关于RouterSploit的内容后，完成并上交**拓展思考**的内容。

2. 实验原理

RouterSploit采用模块化设计，所有模块都继承自基础类，并遵循特定的接口规范。主要模块类型包括：

1. **Exploits模块**：用于漏洞利用
 - 继承自 Exploit 基类
 - 必须实现 run() 和 check() 方法
 - 可以定义模块特定的选项
2. **Scanners模块**：用于漏洞扫描
 - 继承自 Scanner 基类
 - 必须实现 run() 方法
 - 用于检测目标是否存在特定漏洞
3. **Creds模块**：用于凭证管理
 - 继承自 Creds 基类
 - 用于存储和管理认证信息
4. **Generic模块**：用于通用功能
 - 继承自 Generic 基类
 - 提供辅助功能

3. 实验环境

3.1 开发环境要求

- Python 3.6+
- RouterSploit源码
- 代码编辑器（推荐VSCode或PyCharm）
- Git版本控制

3.2 开发环境配置

1. 获取RouterSploit源码：

```
git clone https://github.com/threat9/routersploit
cd routersploit
```

2. 安装开发依赖：

```
pip3 install -r requirements.txt
pip3 install -r requirements-dev.txt # 开发依赖
```

4. 模块开发步骤

4.1 模块结构设计

1. 模块文件位置：

```
routersploit/
├── modules/
│   ├── exploits/
│   │   └── routers/
│   │       ├── vendor/ # 厂商目录
│   │       └── model_exploit.py # 模块文件
```

2. 基本模块模板：

```
#!/usr/bin/env python3

from routersploit.core.exploit import *
from routersploit.core.http.http_client import HTTPClient

class Exploit(HTTPClient):
    __info__ = {
        "name": "模块名称",
        "description": "模块描述",
        "authors": [
            "作者信息",
        ],
        "references": [
            "参考链接",
        ],
        "devices": [
            "受影响的设备型号",
        ],
    }

    # 模块选项定义
    target = OptIP("", "目标IP地址")
    port = OptPort(80, "目标端口")

    def __init__(self):
        super().__init__()
        # 初始化代码

    def check(self):
        # 漏洞检测逻辑
        pass

    def run(self):
        # 漏洞利用逻辑
        pass
```

4.2 模块选项定义

1. 常用选项类型：

```
# IP地址选项
target = OptIP("", "目标IP地址")

# 端口选项
port = OptPort(80, "目标端口")

# 字符串选项
path = OptString("/path", "路径")

# 布尔选项
ssl = OptBool(False, "是否使用SSL")

# 整数选项
timeout = OptInteger(30, "超时时间")

# 选择选项
method = OptSelection(["GET", "POST"], "HTTP方法")
```

2. 选项使用示例：

```
def run(self):
    if self.ssl:
        self.port = 443
    response = self.http_request(
        method="GET",
        path=self.path,
        timeout=self.timeout
    )
```

4.3 模块命令交互设计

1. 命令选项组设计：

```

class Exploit(HTTPClient):
    __info__ = {
        "name": "Router Samba Command Injection",
        "description": "Module exploits command injection vulnerability in router's S",
        "authors": [
            "Your Name",
        ],
        "references": [
            "https://example.com/cve-reference",
        ],
        "devices": [
            "Vulnerable Router Models",
        ],
    }

    # 目标选项组
    target = OptIP("", "Target IPv4 or IPv6 address")
    port = OptPort(80, "Target HTTP port")
    ssl = OptBool(False, "SSL enabled: true/false")

    # 模块特定选项组
    verbosity = OptBool(True, "Verbosity enabled: true/false")
    path = OptString("/goform/SetSambaCfg", "Path to vulnerable endpoint")
    lhost = OptIP("", "Local IP for reverse connection")
    lport = OptPort(8888, "Local port for reverse connection")

```

2. 选项组分类说明：

```

# 目标选项组 (Target options)
# 用于定义目标设备的基本信息
target = OptIP("", "Target IPv4 or IPv6 address") # 目标IP地址
port = OptPort(80, "Target HTTP port") # 目标端口
ssl = OptBool(False, "SSL enabled: true/false") # SSL选项

# 模块选项组 (Module options)
# 用于定义模块特定的配置选项
verbosity = OptBool(True, "Verbosity enabled: true/false") # 详细输出选项
path = OptString("/path", "Path to vulnerable endpoint") # 漏洞路径
lhost = OptIP("", "Local IP for reverse connection") # 本地IP
lport = OptPort(8888, "Local port for reverse connection") # 本地端口

```

3. 选项验证和默认值：

```

def __init__(self):
    super().__init__()

    # 设置选项默认值
    self.target = ""
    self.port = 80
    self.ssl = False

    # 添加选项验证
    self.register_option(
        OptIP("", "Target IPv4 or IPv6 address"),
        required=True, # 必填选项
        validator=self._validate_target # 自定义验证函数
    )

def _validate_target(self, value):
    """验证目标IP地址格式"""
    if not value:
        return False, "Target IP address is required"
    return True, ""

```

4. 命令交互实现：

```

def run(self):
    # 检查必填选项
    if not self.target:
        print_error("Target IP address is required")
        return

    # 显示当前配置
    print_status(f"Target: {self.target}")
    print_status(f"Port: {self.port}")
    print_status(f"SSL: {self.ssl}")

    # 执行漏洞利用
    try:
        response = self.http_request(
            method="POST",
            path=self.path,
            ssl=self.ssl,
            timeout=30
        )

        if response and response.status_code == 200:
            print_success("Exploit completed successfully")
        else:
            print_error("Exploit failed")

    except Exception as e:
        print_error(f"Error: {str(e)}")

```

5. 选项帮助信息:

```

def show_options(self):
    """显示模块选项帮助信息"""
    print("\nTarget options:")
    print("  target      - Target IP address")
    print("  port        - Target port")
    print("  ssl         - Use SSL connection")

    print("\nModule options:")
    print("  verbosity   - Show detailed output")
    print("  path        - Path to vulnerable endpoint")
    print("  lhost       - Local IP for reverse connection")
    print("  lport       - Local port for reverse connection")

```

4.4 模块测试

1. 单元测试编写：

```
# tests/modules/exploits/routers/vendor/test_model_exploit.py
import unittest
from routersploit.modules.exploits.routers.vendor.model_exploit import Exploit

class TestExploit(unittest.TestCase):
    def setUp(self):
        self.exploit = Exploit()

    def test_check(self):
        self.assertFalse(self.exploit.check())

    def test_run(self):
        self.exploit.target = "127.0.0.1"
        self.exploit.port = 80
        self.exploit.run()
```

2. 运行测试：

```
python3 -m pytest tests/modules/exploits/routers/vendor/test_model_exploit.py -v
```

4.5 模块文档编写

1. 模块信息文档：


```
__info__ = {
    "name": "模块名称",
    "description": """
详细的模块描述, 包括:
1. 漏洞类型
2. 影响范围
3. 利用方法
4. 注意事项
""",
    "authors": [
        "作者名称 <email@example.com>",
    ],
    "references": [
        "CVE-2023-XXXX",
        "https://example.com/vulnerability",
    ],
    "devices": [
        "设备型号1",
        "设备型号2",
    ],
}
```

2. 使用说明文档:

```
def show_info(self):
    """显示模块使用说明"""
    print("\n使用方法:")
    print("1. 设置目标IP地址")
    print("    set target 192.168.1.1")
    print("2. 设置目标端口")
    print("    set port 80")
    print("3. 设置本地监听IP")
    print("    set lhost 192.168.1.100")
    print("4. 设置本地监听端口")
    print("    set lport 4444")
    print("5. 执行漏洞利用")
    print("    run")
```

5. 官方文档参考

5.1 核心文档

1. 开发指南

- 模块结构规范
- 最佳实践指南

5.2 实用资源

1. 示例模块

- 参考现有模块
- 学习模块结构
- 了解实现方式

2. 贡献指南

- 代码规范
- 提交规范
- 测试要求

6. 参考资料

1. [RouterSploit官方文档](#)
2. [Python模块开发最佳实践](#)
3. [HTTP客户端开发指南](#)
4. [单元测试编写指南](#)

7. 拓展思考

以下问题可以帮助你更深入地理解RouterSploit模块开发，所有答案都可以在官方文档中找到：

1. 模块继承关系：

- 问题：RouterSploit中的 `Exploit` 、 `Scanner` 、 `Creds` 和 `Generic` 模块分别继承自哪些基类？它们各自有什么特点和使用场景？
- 参考： [RouterSploit Wiki](#)

2. 高级选项类型：

- 问题：除了基本的 `OptIP` 、 `OptPort` 等选项类型外，RouterSploit还支持哪些高级选项类型？如何实现自定义选项类型？
- 参考： [RouterSploit Wiki](#)

3. 模块间通信：

- 问题：在RouterSploit中，如何实现不同模块之间的数据共享和通信？有哪些内置的机制可以支持模块间的协作？
- 参考： [RouterSploit Wiki](#)